Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Main.cs

```
using System;
using System.Linq;

class Program
{
    static void Main(string[] args)
    {
        Product[] products =
        {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mobile", "Electronics"),
            new Product(103, "Shoes", "Fashion"),
            new Product(104, "Watch", "Accessories"),
            new Product(105, "Headphones", "Electronics")
        };

        string productName = "Headphones";

        Product query = LinearSearch(productName, products);

        if (query == null)
        {
            Console.WriteLine("Searched product is not there");
        }
        else
        {
            Console.WriteLine("Product Id: " + query.ProductId);
            Console.WriteLine("Product Name: " + query.ProductName);
            Console.WriteLine("Product Category: " + query.Category);
        }

        Console.WriteLine("\nWith Binary Search");

        Product bsQuery = BinarySearch(productName, products);

        if (bsQuery == null)
        {
            Console.WriteLine("Searched product is not there");
        }
        else
        {
            Console.WriteLine("Product Id: " + bsQuery.ProductId);
            Console.WriteLine("Product Name: " + bsQuery.ProductName);
            Console.WriteLine("Product Category: " + bsQuery.Category);
        }
    }
```

```java
public static Product LinearSearch(String productName, Product[] products){
        int i;
        for(i=0;i<products.length;i++){
            if(products[i].productName.equals(productName))
                return products[i];
        }
        return null;
    }

    public static Product binarySearch(String name, Product[] products) {
        Arrays.sort(products,
                (a, b) -> a.productName.compareTo(b.productName));
        int left = 0;
        int right = products.length - 1;

        while (left <= right) {
            int mid = (left + right) / 2;

            int compare = products[mid].productName.compareTo(name);

            if (compare == 0) {
                return products[mid];
            }
            else if (compare < 0) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }

        return null;
    }
```

Product.cs

```java
package DataStructures;

public class Product {
    int productId;
    String productName;
    String category;
    public Product(int pdId, String pdName, String catgry){
        this.productId = pdId;
        this.productName = pdName;
        this.category = catgry;
    }
}
```

Output:

Product Id: 105
Product Name: Headphones
Product Category: Electronics

With Binary Search
Product Id: 105
Product Name: Headphones
Product Category: Electronics

Time complexity analysis

Linear Search:
Best Case: O(1)
Average Case: O(n)
Worst Case: O(n)

Binary Search:
Best Case: O(1)

Average Case: O(log n)
Worst Case: O(log n)